

Pipeline allelic frequency - ref polarization

III. SNP calling

The SNP calling included the bam file of the two parents MIL2 - KAD1 based on the mapping on Col0 reference genome.

1. Call the SNP

run the script: `snp_calling.sh`

```
#!/bin/bash -l

#####
# SNP calling script
# Script from Tahir Ali, modified by Justine Floret
# 24/11/2025
#####

#SBATCH --nodes=1
#SBATCH --time=72:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=40gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/bcftools_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/bcftools_%J.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0
ulimit -n 2003

# Define directories
bam_dir="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping_MIL2/"
output_vcf="/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0"

# Ensure output directory exists
mkdir -p ${output_vcf}

# Run bcftools mpileup on all BAM files in the directory
bcftools mpileup \
  -a AD,DP,SP \
  -q 40 \
  -Q 30 \
  -Ou \
  -f /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas \
  -b /projects/ag-demeaux/jfloret/Ath_Cologne_2025/mapping/bam_list.txt | bcftools call \
  -m -v \
  -a GQ,GP \
  -Oz \
  -o /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/ath_Cologne_refCol0_B1_2025.vcf.gz
```

2. Soft filtering

`snp_quality_SoftFilter`

```

#!/bin/bash
set -euo pipefail

#####
# SNP filtering script
# Script from Tahir Ali, modified by Justine Floret and Margarita Takou
# 12.2025
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=20gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/snpfiltering_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/snpfiltering_%J.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0

# set working directory
cd /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0 # adjust if needed

# INPUT
VCF="ath_Cologne_refCol0_B1_2025.vcf.gz"

echo "=== STARTING VCF FILTERING PIPELINE ==="

#####
# 1. Remove indels, other variants → keep only biallelic SNPs
#####
echo "[1] Filtering SNPs only..."

# -g: filter SNPs within _INT_ base pairs of an indel or other other variant type
# -M2 -v snps to only view biallelic SNPs

bcftools filter -Ou -g 5:indel,other $VCF | \
  bcftools view -Oz -M2 -V indels,other,mnps \
  -o ath_Cologne_refCol0_B1_2025_noindel.vcf.gz

bcftools index ath_Cologne_refCol0_B1_2025_noindel.vcf.gz
echo "SNP count:"
bcftools view -H ath_Cologne_refCol0_B1_2025_noindel.vcf.gz | wc -l

#####
# 2. MQ, QUAL, and Strand Bias (SP) filters
#####
echo "[3] Applying MQ, QUAL, SP filters..."

# MQ: minimum mapping quality
bcftools filter -Oz -e 'MQ<40' ath_Cologne_refCol0_B1_2025_noindel.vcf.gz \
  -o ath_Cologne_refCol0_B1_2025_noindel_MQ40.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40.vcf.gz

# QUAL: site call quality
bcftools filter -Oz -e 'QUAL<30' ath_Cologne_refCol0_B1_2025_noindel_MQ40.vcf.gz \
  -o ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30.vcf.gz

# SP Phred-scaled strand bias P-value
SP_THRESH=50

bcftools filter -Oz -e "SP>$SP_THRESH" ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30.vcf.gz \
  -o ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz

```

```

echo "Remaining sites after SP filter:"
bcftools view -H ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz | wc -l

#####
# 4. Soft filtering (GQ)
#####
echo "[7] Soft filtering: GQ>=25 ..."

bcftools filter -Ou -S . -e 'FMT/GQ<25'
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50.vcf.gz \
-o ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

echo "Sites after soft filtering:"
bcftools view -H ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz | wc -l

echo "=== FILTERING COMPLETE ==="

```

3. Rename the pool in the vcf file

/projects/ag-demeaux/jflorete/Ath_Cologne_2025/Sample_Names.tab

```

325543 113
325545 159
325549 173
325605 F2_1
325606 F2_2
325607 F2_3
A006200066_116050_S98_L004 Mil2
A006200066_116048_S97_L004 Kad1

```

```

# checking the vcf name
bcftools query -l ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz | head

bcftools reheader -s parents_name.txt -o
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25_IDparents.vcf.gz
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools query -l ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25_IDparents.vcf.gz

mv ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25_IDparents.vcf.gz
ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

bcftools index ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz

```

IV. SNP filtering

This time I will filter the entire vcf file with the different filters:

- Non missing GT & heterozygotes in the parents sequences
- SNP in the genic region
- Keep only the position where the DP < 2x mean DP
- SNP segregating in the F2 population

Then I will polarize the vcf file to have the Mil2 allele being the reference allele.

1. No missing GT or heterozygote SNP in the parents genome

First get the index of both parents in the vcf file

```
module load bio/BCFtools/1.19-GCC-13.2.0
bcftools query -l ../ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz | nl -v0
```

```
bcftools view \
-e 'GT[79]="./." || GT[80]="./." || GT[79]="het" || GT[80]="het"' \
../ath_Cologne_refCol0_B1_2025_noindel_MQ40_QUAL30_SP50_GQ25.vcf.gz \
-Oz -o ath_Cologne_refCol0_B1_2025_ParentalFilter.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_ParentalFilter.vcf.gz
```

2. Keep only the SNP in the genic regions

```
bcftools view \
-R /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_genes.bed \
-Oz -o ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions.vcf.gz \
ath_Cologne_refCol0_B1_2025_ParentalFilter.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions.vcf.gz
```

3. Keep only the position where the DP < 2x mean DP

[vcf_filtering_allelic_depth.sh](#)

```
#!/bin/bash
set -euo pipefail

#####
# vcf filtering for analysis script
# Script from Justine Floret
# 12.2025
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=10gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering/vcffiltering_%j.err
#SBATCH --output=/projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering/vcffiltering_%J.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0

# INPUT
VCF="ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions.vcf.gz"

cd /projects/ag-demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering/

echo "=== STARTING VCF FILTERING PIPELINE ==="

#####
# 1. Compute mean DP and number of SNP per chromosome
#####
echo "=== [1] Mean DP per Sample ==="
```

```

bcftools query -f '[%DP\t]\n' "$VCF" > DP_matrix.txt
awk '
{
    if (NR==1) ncol = NF
    for (i=1; i<=NF; i++) if ($i != ".") { sum[i] += $i; n[i]++ }
}
END {
    print "# SampleIndex\tMeanDP"
    for (i=1; i<=ncol; i++) {
        mean = (n[i]>0 ? sum[i]/n[i] : 0)
        print i "\t" sprintf("%.2f", mean)
    }
}' DP_matrix.txt > DP_mean_per_sample.txt

#####
# 2. DP FILTERING
#####
echo "=== [2] Per sample per SNP filtering DP ==="

cp ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions.vcf.gz tmp_DPclean.vcf.gz
bcftools index tmp_DPclean.vcf.gz

# Skip header and read IDX MEAN_DP only
tail -n +2 DP_mean_per_sample.txt | while read -r IDX MEAN_DP; do
    THR=$(awk -v m="$MEAN_DP" 'BEGIN{printf "%.2f", 2*m}')
    echo "Filtering index $IDX with thresholds 10 <= DP <= $THR"

    bcftools +setGT tmp_DPclean.vcf.gz \
        -Oz -o tmp_DPclean2.vcf.gz \
        -- -t q -n . \
        -i "FMT/DP[$IDX]<10 | FMT/DP[$IDX]>$THR"

    mv tmp_DPclean2.vcf.gz tmp_DPclean.vcf.gz
    bcftools index tmp_DPclean.vcf.gz
done

mv tmp_DPclean.vcf.gz ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean.vcf.gz
bcftools index ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean.vcf.gz

echo "=== MAY THE FORCE BE WITH YOU ==="

```

4. Keep the segregating positions

Filters positions if **any** of the three samples fails (missing AD/DP, zero DP, or $AF \notin [0.4, 0.6]$).

Keeps sites where **all three** pass; other samples unchanged

vcf_filtering_segregatingSNP.sh

```

#!/bin/bash
set -euo pipefail

#####
# vcf filtering for analysis script
# Script from Justine Floret
# 12.2025
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=20gb
#SBATCH --account=ag-demeaux

```

```
#SBATCH --error=/projects/ag-
demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering/vcffiltering_%j.err
#SBATCH --output=/projects/ag-
demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering/vcffiltering_%J.out

module purge
module load bio/BCFtools/1.19-GCC-13.2.0

echo "=== Filter sites: AF [0.4,0.6] + coverage in 76,77,78 ==="

bcftools view ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean.vcf.gz \
-i 'FMT/DP[76] > 0 & FMT/AD[76:0]/FMT/DP[76] >=0.4 & FMT/AD[76:0]/FMT/DP[76] <=0.6 |
    FMT/DP[77] > 0 & FMT/AD[77:0]/FMT/DP[77] >=0.4 & FMT/AD[77:0]/FMT/DP[77] <=0.6 |
    FMT/DP[78] > 0 & FMT/AD[78:0]/FMT/DP[78] >=0.4 & FMT/AD[78:0]/FMT/DP[78] <=0.6' \
-Oz -o ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean_segregatingSNP.vcf.gz

bcftools index
ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean_segregatingSNP.vcf.gz

echo "=== MAY THE FORCE BE WITH YOU ==="
```

V. Polarization of the reference allele

1. Tahir Ali script

```
"""
=====
PARENT-BASED REF/ALT SWITCHING & VCF POLARIZATION SCRIPT
=====
Author: Tahir Ali | https://tahirali-biomics.github.io/
Date: 11.02.2026

Adapted from: Kristian Ullrich's polarizeVCFbyOutgroup.py (2021)
               https://github.com/kullrich/bio-
scripts/blob/adfef64a803ca2104c60e68b29e241f6b4776405/vcf/polarizeVCFbyOutgroup.py
Original license: MIT

Modifications and improvements:
    • Added comprehensive FORMAT field flipping (GT, PL, AD, GP)
    • Fixed missing genotype handling (./. sites now properly removed)
    • Enhanced documentation for outgroup indexing
    • Added detailed processing statistics
    • Improved error handling and debugging output
    • Added parent-based polarization workflow description

This script is released under the MIT License (maintained from original work).

PURPOSE:
    This script polarizes a VCF file using a specified outgroup/parent individual.
    When the outgroup is homozygous ALT (1/1), it swaps REF and ALT alleles and
    flips all relevant FORMAT fields across ALL samples. This ensures the outgroup
    becomes homozygous REF (ancestral state) after flipping.

WORKFLOW SUMMARY:
    1. Read VCF line by line
    2. Skip header lines (add AA tag if -add specified)
    3. Extract outgroup genotype (GT field)
    4. Apply filtering logic:
        • Missing GT (./.) → REMOVE site
        • Homozygous REF (0/0) → KEEP site (no change)
        • Heterozygous (0/1,1/0) → REMOVE site (unless -keep)
        • Homozygous ALT (1/1) → SWITCH alleles & FLIP all samples
    5. When switching:
        • Swap REF and ALT columns
```

- Flip GT: 0↔1 for all genotypes
 - Flip PL: swap 0/0 and 1/1 likelihoods
 - Flip AD: swap REF and ALT read counts
 - Flip GP: swap 0/0 and 1/1 posterior probabilities
 - DP, SP, GQ: unchanged (allele-independent)
6. Add Ancestral Allele (AA) tag if -add specified
 7. Write processed line to output

KEY FEATURES:

- Handles phased (|) and unphased (/) genotypes
- Properly flips all FORMAT fields: GT, PL, AD, GP
- Preserves DP, SP, GQ (invariant to allele switching)
- Works with gzipped or plain text VCF
- Adds ancestral allele info to INFO field (optional)
- Keeps heterozygous outgroup sites (optional)

FILTERING BEHAVIOR (based on outgroup genotype):

Outgroup GT	Default (-keep)	With -keep
./., .	REMOVE	REMOVE
0/0	KEEP	KEEP
0/1,1/0	REMOVE	KEEP
1/1	SWITCH & KEEP	SWITCH & KEEP

OUTGROUP INDEXING (CRITICAL!):

The -ind parameter expects a 0-based SAMPLE INDEX (position after FORMAT), NOT the absolute column number.

CORRECT: -ind 80 (80th sample after FORMAT column)

WRONG: -ind 88 (absolute column position)

TO FIND CORRECT INDEX:

```
zgrep "^#CHROM" your.vcf.gz | head -1 | tr '\t' '\n' | nl -v 0 | grep "OutgroupName"
```

OUTPUT STATISTICS:

- Parsed: Total sites processed
- Removed: Sites with missing outgroup data
- Kept: Homozygous REF and flipped sites
- Removed: Heterozygous sites (unless -keep)
- Switched: Sites where REF/ALT were swapped
- Total written: Sites in output VCF

EXAMPLE USAGE:

```
# Basic polarization (remove missing/heterozygous, add AA tag)
python polarizeVCF.py -vcf input.vcf.gz -out polarized.vcf.gz -ind 80 -add
```

```
# Keep heterozygous outgroup sites
python polarizeVCF.py -vcf input.vcf -out polarized.vcf -ind 80 -keep -add
```

```
# No ancestral allele annotation
python polarizeVCF.py -vcf input.vcf.gz -out polarized.vcf.gz -ind 80
```

NOTE:

This script is designed for BI-ALLELIC SNPs only. Multi-allelic sites and indels should be filtered out before running.

```
=====
"""
```

```
#!/usr/bin/python
# -*- coding: UTF-8 -*-
```

```
"""
```

```
=====
PARENT-BASED REF/ALT SWITCHING & VCF POLARIZATION SCRIPT
```

```
=====
Author: Tahir Ali | https://tahirali-biomics.github.io/
Date: 11.02.2026
```

```
Adapted from: Kristian Ullrich's polarizeVCFbyOutgroup.py (2021)
               https://github.com/kullrich/bio-
scripts/blob/adfef64a803ca2104c60e68b29e241f6b4776405/vcf/polarizeVCFbyOutgroup.py
Original license: MIT
```

Modifications and improvements:

- Added comprehensive FORMAT field flipping (GT, PL, AD, GP)
- Fixed missing genotype handling (./. sites now properly removed)
- Enhanced documentation for outgroup indexing
- Added detailed processing statistics
- Improved error handling and debugging output
- Added parent-based polarization workflow description

This script is released under the MIT License (maintained from original work).

PURPOSE:

This script polarizes a VCF file using a specified outgroup/parent individual. When the outgroup is homozygous ALT (1/1), it swaps REF and ALT alleles and flips all relevant FORMAT fields across ALL samples. This ensures the outgroup becomes homozygous REF (ancestral state) after flipping.

```
"""
```

```
import sys
import argparse
import textwrap
import gzip
import re
```

```
def multiple_replace(string, rep_dict):
    """Replace multiple patterns in string using dictionary mapping."""
    pattern = re.compile("|".join([re.escape(k) for k in sorted(rep_dict, key=len,
reverse=True)]), flags=re.DOTALL)
    return pattern.sub(lambda x: rep_dict[x.group(0)], string)
```

```
def flip_genotype(gt):
    """
    Flip genotype 0↔1 while preserving phasing.
    Examples:
        0/0 → 1/1, 0|0 → 1|1
        1/1 → 0/0, 1|1 → 0|0
        0/1 → 1/0, 0|1 → 1|0
        1/0 → 0/1, 1|0 → 0|1
        ./ . → ./ . (unchanged)
    """
    if gt == '0/0' or gt == '0|0':
        return '1/1' if '/' in gt else '1|1'
    elif gt == '1/1' or gt == '1|1':
        return '0/0' if '/' in gt else '0|0'
    elif gt == '0/1' or gt == '0|1':
        return '1/0' if '/' in gt else '1|0'
    elif gt == '1/0' or gt == '1|0':
        return '0/1' if '/' in gt else '0|1'
    else:
        return gt # ./ ., .|. , etc.
```

```
def flip_pl_values(pl_str):
    """
    Flip PL (Phred-scaled genotype likelihood) values.
    FORMAT: PL[0]=0/0, PL[1]=0/1, PL[2]=1/1
    After swap: PL[0]=1/1, PL[1]=0/1, PL[2]=0/0
    """
```



```

if pl_str == '.':
    return pl_str
pl_values = pl_str.split(',')
if len(pl_values) == 3: # For bi-allelic sites
    # Swap PL[0] (0/0) with PL[2] (1/1), keep PL[1] (0/1) the same
    return ','.join([pl_values[2], pl_values[1], pl_values[0]])
return pl_str

def flip_ad_values(ad_str):
    """
    Flip AD (Allelic depths) values.

    FORMAT: AD[0]=REF reads, AD[1]=ALT reads
    After swap: AD[0]=ALT reads, AD[1]=REF reads
    """
    if ad_str == '.':
        return ad_str
    ad_values = ad_str.split(',')
    if len(ad_values) == 2: # For bi-allelic sites
        # Swap AD[0] (REF) with AD[1] (ALT)
        return ','.join([ad_values[1], ad_values[0]])
    return ad_str

def flip_gp_values(gp_str):
    """
    Flip GP (Genotype posterior probability) values.

    FORMAT: GP[0]=0/0, GP[1]=0/1, GP[2]=1/1
    After swap: GP[0]=1/1, GP[1]=0/1, GP[2]=0/0
    """
    if gp_str == '.':
        return gp_str
    gp_values = gp_str.split(',')
    if len(gp_values) == 3: # For bi-allelic sites
        # Swap GP[0] (0/0) with GP[2] (1/1), keep GP[1] (0/1) the same
        return ','.join([gp_values[2], gp_values[1], gp_values[0]])
    return gp_str

def flip_format_fields(format_str, sample_str):
    """
    Master function to flip all relevant FORMAT fields when alleles are swapped.
    Fields flipped:
        - GT: 0↔1 genotype swap
        - PL: 0/0 ↔ 1/1 likelihood swap
        - AD: REF ↔ ALT read count swap
        - GP: 0/0 ↔ 1/1 probability swap
    Fields unchanged (allele-independent):
        - DP: Total depth
        - SP: Strand bias p-value
        - GQ: Genotype quality
    """
    if format_str == '.' or sample_str == '.':
        return format_str, sample_str

    format_fields = format_str.split(':')
    sample_fields = sample_str.split(':')

    # Create a dictionary for easier access
    sample_dict = dict(zip(format_fields, sample_fields))

    # Flip each field as needed
    if 'GT' in sample_dict:
        sample_dict['GT'] = flip_genotype(sample_dict['GT'])

    if 'PL' in sample_dict:

```

```

        sample_dict['PL'] = flip_pl_values(sample_dict['PL'])

    if 'AD' in sample_dict:
        sample_dict['AD'] = flip_ad_values(sample_dict['AD'])

    if 'GP' in sample_dict:
        sample_dict['GP'] = flip_gp_values(sample_dict['GP'])

    # DP, SP, and GQ remain unchanged when swapping alleles
    # (they are invariant to which allele is REF vs ALT)

    # Reconstruct the sample string
    new_sample_fields = [sample_dict.get(field, sample_fields[i] if i < len(sample_fields) else
    '.')
                        for i, field in enumerate(format_fields)]

    return format_str, ':'.join(new_sample_fields)

def parse_lines(fin, fou, ind, keep, add):
    """
    Main VCF parsing and processing function.

    Steps for each non-header line:
    1. Extract outgroup genotype
    2. Check for missing data → skip if found
    3. Case 0/0: Keep site (add AA tag if requested)
    4. Case 0/1,1/0: Skip unless -keep specified
    5. Case 1/1: Swap REF/ALT, flip all FORMAT fields for ALL samples
    6. Write processed line
    """
    switchcount = 0
    removecount = 0
    outmissing = 0
    totalcount = 0
    kept_heterozygous = 0
    kept_hom_ref = 0

    for line in fin:
        # ----- HEADER PROCESSING -----
        if line[0] == '#':
            if add:
                if line.split('\t')[0] == '#CHROM':
                    fou.write('##INFO=<ID=AA,Number=1,Type=String,Description="Ancestral
Allele">\n')
                    fou.write(line)
                else:
                    fou.write(line)
            else:
                fou.write(line)
            continue

        # ----- DATA LINE PROCESSING -----
        totalcount += 1
        linesplit = line.strip().split('\t')

        # Check if outgroup column exists
        if len(linesplit) <= 8 + ind:
            outmissing += 1
            continue

        # Extract outgroup information
        format_col = linesplit[8]
        sample_col = linesplit[8 + ind]

        # Get genotype from GT field
        gt_field = None
        if format_col != '.' and sample_col != '.':

```

```

try:
    format_fields = format_col.split(':')
    if 'GT' in format_fields:
        gt_index = format_fields.index('GT')
        sample_fields = sample_col.split(':')
        if gt_index < len(sample_fields):
            gt_field = sample_fields[gt_index]
except:
    pass

# DEBUG: Uncomment to verify outgroup genotypes
# print(f"DEBUG: Format: {format_col}, Sample: {sample_col}, GT: {gt_field}")

# ----- MISSING DATA CHECK -----
if gt_field is None or gt_field == '.' or gt_field.startswith('.') or '..' in gt_field:
    outmissing += 1
    continue # Skip sites with missing outgroup data

# ----- CASE 1: HOMOZYGOUS REF -----
if gt_field == '0/0' or gt_field == '0|0':
    kept_hom_ref += 1
    if add:
        linesplit[7] = 'AA=' + linesplit[3] + ';' + linesplit[7]
        fou.write('\t'.join(linesplit) + '\n')
    else:
        fou.write(line)

# ----- CASE 2: HETEROZYGOUS -----
elif gt_field in ['0/1', '1/0', '0|1', '1|0']:
    removecount += 1
    if keep:
        kept_heterozygous += 1
        fou.write(line)
    # If not keep, skip heterozygous sites

# ----- CASE 3: HOMOZYGOUS ALT - SWITCH NEEDED -----
elif gt_field == '1/1' or gt_field == '1|1':
    switchcount += 1

    # Swap REF and ALT alleles
    REF = linesplit[4]
    ALT = linesplit[3]
    linesplit[3] = REF
    linesplit[4] = ALT

    # Flip FORMAT fields for ALL samples (not just outgroup)
    format_col = linesplit[8]
    for i in range(9, len(linesplit)):
        if linesplit[i] != '.':
            format_col, linesplit[i] = flip_format_fields(format_col, linesplit[i])

    if add:
        linesplit[7] = 'AA=' + linesplit[3] + ';' + linesplit[7]
        fou.write('\t'.join(linesplit) + '\n')
    else:
        fou.write('\t'.join(linesplit) + '\n')

# ----- CASE 4: OTHER (multi-allelic, etc.) -----
else:
    removecount += 1
    if keep:
        fou.write(line)
    # If not keep, skip these sites

# ----- FINAL STATISTICS -----
print('\n' + '='*60)
print('PROCESSING SUMMARY')
print('='*60)

```

```

print(f'Total sites parsed:          {totalcount:8d}')
print(f'Removed (missing outgroup): {outmissing:8d}')
print(f'Kept (homozygous REF):       {kept_hom_ref:8d}')
if keep:
    print(f'Kept (heterozygous):      {kept_heterozygous:8d}')
    print(f'Removed (other):          {removecount - kept_heterozygous:8d}')
else:
    print(f'Removed (heterozygous/other): {removecount:8d}')
print(f'Switched (REF↔ALT):           {switchcount:8d}')
print(f'Total sites written:          {kept_hom_ref + switchcount + (kept_heterozygous if keep
else 0):8d}')
print('='*60)

def main():
    parser = argparse.ArgumentParser(
        prog='polarizeVCFbyOutgroup',
        description='''\n
        Parent-based REF/ALT switching and VCF polarization using outgroup genotype.

        =====
        QUICK START:
        1. Find your outgroup index:
            zgrep "^#CHROM" input.vcf.gz | tr '\t' '\n' | nl -v 0 | grep "Outgroup"

        2. Run polarization:
            python %(prog)s -vcf input.vcf.gz -out polarized.vcf.gz -ind 80 -add
        =====
        ''',
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog='''\n
        EXAMPLES:
        # Basic usage with gzipped files
        python polarizeVCFbyOutgroup.py -vcf variants.vcf.gz -out polarized.vcf.gz -ind 80 -add

        # Keep heterozygous outgroup sites
        python polarizeVCFbyOutgroup.py -vcf variants.vcf -out polarized.vcf -ind 80 -keep -add

        # No ancestral allele annotation
        python polarizeVCFbyOutgroup.py -vcf variants.vcf.gz -out polarized.vcf.gz -ind 80

        CITATION:
        Adapted from Kristian Ullrich (2021)
        ''')

    parser.add_argument('-vcf', required=True, help='Input VCF file (can be .gz)')
    parser.add_argument('-out', required=True, help='Output VCF file (can be .gz)')
    parser.add_argument('-ind', type=int, required=True,
        help='0-based SAMPLE INDEX of outgroup/parent (NOT column number!)')
    parser.add_argument('-keep', action='store_true',
        help='Keep sites where outgroup is heterozygous')
    parser.add_argument('-add', action='store_true',
        help='Add Ancestral Allele (AA) tag to INFO field')

    args = parser.parse_args()

    print('\n' + '='*60)
    print('PARENT-BASED REF/ALT SWITCHING')
    print('='*60)
    print(f'Input VCF: {args.vcf}')
    print(f'Output VCF: {args.out}')
    print(f'Outgroup index: {args.ind} (0-based sample position)')
    print(f'Keep heterozygous: {args.keep}')
    print(f'Add AA tag: {args.add}')
    print('='*60 + '\n')

```

```
# Handle gzipped or plain text files
if args.out.endswith('gz'):
    with gzip.open(args.out, 'wt') as fou:
        if args.vcf.endswith('gz'):
            with gzip.open(args.vcf, 'rt') as fin:
                parse_lines(fin, fou, args.ind, args.keep, args.add)
        else:
            with open(args.vcf, 'rt') as fin:
                parse_lines(fin, fou, args.ind, args.keep, args.add)
else:
    with open(args.out, 'wt') as fou:
        if args.vcf.endswith('gz'):
            with gzip.open(args.vcf, 'rt') as fin:
                parse_lines(fin, fou, args.ind, args.keep, args.add)
        else:
            with open(args.vcf, 'rt') as fin:
                parse_lines(fin, fou, args.ind, args.keep, args.add)

if __name__ == '__main__':
    main()
```

2. Test on subdata set

```
bcftools view -Oz -o test_before_polarization.vcf.gz \
-s 113,159,173,F2_1,F2_2,F2_3,Kad1,Mil2 \
ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPClean_seggregatingSNP.vcf.gz

bcftools index test_before_polarization.vcf.gz
```

```
awk -F'\t' '$1=="Chr4" && $2>=260000 && $2<=280000' test_before_polarization_RfriendlyAF.tsv |
less
awk -F'\t' '$1=="Chr4" && $2==269180' test_before_polarization_RfriendlyAF.tsv | less
q
```

```
python3 ../../Script/flip_mil2_full_vcf_2.py \
-vcf test_before_polarization.vcf.gz \
-out test_after_polarization.vcf.gz \
-ind 8 -add
```

```
=====
PARENT-BASED REF/ALT SWITCHING
=====
Input VCF:  test_before_polarization.vcf.gz
Output VCF: test_after_polarization.vcf.gz
Outgroup index: 8 (0-based sample position)
Keep heterozygous: False
Add AA tag: True
=====

=====
PROCESSING SUMMARY
=====
Total sites parsed:          161036
Removed (missing outgroup):   1938
Kept (homozygous REF):        80420
Removed (heterozygous/other): 0
Switched (REF↔ALT):          78678
Total sites written:         159098
=====
```

Extract the allelic frequency

```
bcftools query \
-f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE\t%GT\t%AD\t%DP]\n' \
test_after_polarization.vcf.gz \
| awk '
{
    chrom=$1; pos=$2; ref=$3; alt=$4;
    # fields from 5 onwards are groups of 4: SAMPLE, GT, AD, DP
    for (i=5; i<=NF; i+=4) {
        sample=$(i); gt=$(i+1); ad=$(i+2); dp=$(i+3);
        altfreq="NA";
        if (ad != "." && dp != "." && dp > 0) {
            split(ad,a,","); # a[1]=REF reads, a[2]=ALT reads (for biallelic)
            altfreq = a[2] / dp;
        }
        print chrom"\t"pos"\t"ref"\t"alt"\t"sample"\t"gt"\t"ad"\t"dp"\t"altfreq;
    }
}
' > test_after_polarization_RfriendlyAF.tsv
# Column header: CHROM POS REF ALT SAMPLE GT AD DP ALTFREQ
```

3. Run on the complete data set

[vcf_polarized_reference.sh](#)

```
#!/bin/bash
set -euo pipefail

#####
# Polarized the reference allele
# Script from Tahir Ali
# 12.2025
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=10gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-demeaux/jflorete/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering/vcffiltering_%j.err
#SBATCH --output=/projects/ag-demeaux/jflorete/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering/vcffiltering_%J.out

module purge
module load lang/Python/3.11.5-GCCcore-13.2.0
module load bio/BCFtools/1.19-GCC-13.2.0

cd /projects/ag-demeaux/jflorete/Ath_Cologne_2025/SNP_calling/vcf_refCol0/SNP_filtering

python3 ../../Script/flip_mil2_full_vcf_2.py \
-vcf ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPClean_segSegatingSNP.vcf.gz \
-out
ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPClean_segSegatingSNP_polarizedRef.vcf.gz \
-ind 81 -add

gunzip
ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPClean_segSegatingSNP_polarizedRef.vcf.gz
bgzip -f
ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPClean_segSegatingSNP_polarizedRef.vcf
bcftools index
ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPClean_segSegatingSNP_polarizedRef.vcf.gz
```

```
echo "=== MAY THE FORCE BE WITH YOU ==="
```

```
=====
PARENT-BASED REF/ALT SWITCHING
=====
Input VCF:  ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean_segregatingSNP.vcf.gz
Output VCF:
ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean_segregatingSNP_polarizedRef.vcf.gz
Outgroup index: 81 (0-based sample position)
Keep heterozygous: False
Add AA tag: True
=====

=====
PROCESSING SUMMARY
=====
Total sites parsed:          161036
Removed (missing outgroup):   1938
Kept (homozygous REF):        80420
Removed (heterozygous/other): 0
Switched (REF↔ALT):           78678
Total sites written:          159098
=====
```

V. Median allelic frequency on 50,000 bp windows

- Map AFs and keep all values per window, then Derive number of SNPs and number of NAs
Computes the median AF and also a comma-separated list of all AF values overlapping each window

```
echo "=== Extracting the allelic frequency of the F5 pools in R friendly table ==="

bcftools query \
  -f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE\t%GT\t%AD\t%DP]\n' \

ath_Cologne_refCol0_B1_2025_ParentalFilter_GeneRegions_DPclean_segregatingSNP_polarizedRef.vcf.gz \
  | awk '
  {
    chrom=$1; pos=$2; ref=$3; alt=$4;
    # fields from 5 onwards are groups of 4: SAMPLE, GT, AD, DP
    for (i=5; i<=NF; i+=4) {
      sample=$(i); gt=$(i+1); ad=$(i+2); dp=$(i+3);
      altfreq="NA";
      if (ad != "." && dp != "." && dp > 0) {
        split(ad,a,","); # a[1]=REF reads, a[2]=ALT reads (for biallelic)
        altfreq = a[2] / dp;
      }
      print chrom"\t"pos"\t"ref"\t"alt"\t"sample"\t"gt"\t"ad"\t"dp"\t"altfreq;
    }
  }
' > ath_Cologne_refCol0_B1_2025_polarizedRef_AF_Rfriendly.tsv
# Column header: CHROM POS REF ALT SAMPLE GT AD DP ALTFREQ
```

median_frequency_per_50kb_windows.sh

```
#!/bin/bash
set -euo pipefail
```

```

#####
# Median allelic frequency on 50,000 bp windows
# Script from Justine Floret
# 01.2026
#####

#SBATCH --nodes=1
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=10gb
#SBATCH --account=ag-demeaux
#SBATCH --error=/projects/ag-
demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/F5_allelic_frq_50kbwindows/vcffiltering_%
j.err
#SBATCH --output=/projects/ag-
demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/F5_allelic_frq_50kbwindows/vcffiltering_%
J.out

module purge
module load bio/BEDTools/2.31.1-GCC-13.2.0

# set working directory
cd /projects/ag-
demeaux/jfloret/Ath_Cologne_2025/SNP_calling/vcf_refCol0/F5_allelic_frq_50kbwindows

#####
# 1. Get chromosome lengths
#####
echo "=== Get chromosome lengths ==="

cut -f1,2 /projects/ag-demeaux/jfloret/Ath_Cologne_2025/Ref_genome/TAIR10_chr_all.fas.fai | grep -
E '^Chr[1-5]' > chr_lengths.txt

#####
# 2. Create 50000 bp windows for each chromosome
#####
echo "=== Create 50000 bp windows for each chromosome ==="

bedtools makewindows -g chr_lengths.txt -w 50000 > windows_50kb.bed

#####
# 3. Create per sample Bed Allelic frequency table
#####
echo "=== Create per sample Bed Allelic frequency table ==="

while read SAMPLE; do
    tail -n +2 ../ath_Cologne_refCol0_B1_2025_polarizedRef_AF_Rfriendly.tsv \
    | awk -v S="$SAMPLE" 'BEGIN{OFS="\t"} $5==S {print $1, $2-1, $2, $9}' \
    > "${SAMPLE}.AF.bed"
done < ../F5_list.txt

#####
# 4. Compute the median AF per window, per sample, per chromosome
#####
echo "=== Compute the median AF per window ==="

while read SAMPLE; do
    bedtools map \
    -a windows_50kb.bed \
    -b "${SAMPLE}.AF.bed" \
    -c 4,4 \
    -o median,collapse \
    > "${SAMPLE}.AF_50kb_median_raw.bed"

    awk 'BEGIN{FS=OFS="\t"}
{

```



```

af_list = $5
if (af_list == ".") {print $1,$2,$3,$4,0,0; next}
n_total = split(af_list, a, ",")
n_na = 0
for (i=1; i<=n_total; i++) if (a[i]=="NA" || a[i]==".") n_na++
print $1, $2, $3, $4, n_total, n_na
}' "${SAMPLE}.AF_50kb_median_raw.bed" > "${SAMPLE}.AF_50kb_median.bed"
done < ../F5_list.txt

#####
# 5. Merged in one table
#####
echo "=== Merged in one table ==="

while read SAMPLE; do
  awk -v S="${SAMPLE}" 'BEGIN{OFS="\t"} {center = int(($2+$3)/2); print $1, center, S, $4, $5, $6}' \
    "${SAMPLE}.AF_50kb_median.bed"
done < ../F5_list.txt > F5_pools_50kb_medianAF.tsv

echo "=== May the force be with you ==="

```

[\\${SAMPLE}.AF_50kb_median.bed](#)

col1: chromosome

col2: window start (0-based)

col3: window end

col4: median AF

col5: number of SNP records in the window (`n_total`)

[F5_pools_50kb_medianAF.tsv](#)

col 1: Chr,

col 2: Pos middle of the window

Col 3: Sample_ID

Col 4: median_AF

Col 5: SNP_tot

```

echo "=== Extracting the allelic frequency of the F5 pools in R friendly table ==="

bcftools query \
  -f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE\t%GT\t%AD\t%DP]\n' \
  ath_Cologne_refCol0_B1_2025.vcf.gz \
  | awk '
  {
    chrom=$1; pos=$2; ref=$3; alt=$4;
    # fields from 5 onwards are groups of 4: SAMPLE, GT, AD, DP
    for (i=5; i<=NF; i+=4) {
      sample=$(i); gt=$(i+1); ad=$(i+2); dp=$(i+3);
      altfreq="NA";
      if (ad != "." && dp != "." && dp > 0) {
        split(ad,a,","); # a[1]=REF reads, a[2]=ALT reads (for biallelic)
        altfreq = a[2] / dp;
      }
      print chrom"\t"pos"\t"ref"\t"alt"\t"sample"\t"gt"\t"ad"\t"dp"\t"altfreq;
    }
  }
' > ath_Cologne_refCol0_B1_2025AF_Rfriendly.tsv
# Column header: CHROM POS REF ALT SAMPLE GT AD DP ALTFREQ

```